

url4 topology — endpoint, node, discovery, addressing, transport — open work

Questions, sketches, spec deltas: the part still moving

STATUS	proposed — owner review; one decision left open in §5; aligned with the url4-refactor drafts 2026-09-08 (§12); vocabulary realigned to the spec's Node/Endpoint
WORK ITEM	OME-1110
DATE	2026-09-04
OWNER	Sergey Bershadsky (execution and telemetry architecture)
GRAMMAR OWNER	Kevin McDonough (URL4 spec, Parts A/B)
SPEC POINTERS	URL4.ai Specification v0.5 DRAFT — Parts A/B (main) and Parts C–I drafts (url4-refactor branch)

Read the core document first; this part assumes its vocabulary.

What this part is

The companion to the core document. It carries what is not yet decided: the questions on authentication (§10), the networking sketch (§11), the alignment with Kevin's spec (§12), the peer-to-peer direction (§13), the spec deltas for Kevin (Appendix A), the follow-up work items (Appendix B) and the vocabulary crosswalk (Appendix D). Read the core first; every term used here is defined there.

10. Deferred, and two questions to move forward

Authentication as a node concern (question, not a decision). Keeping node and endpoint distinct makes a safe place for authentication possible, because it gives the credentials exactly one owner. The spec's inter-node auth (Part H §31, "under active development") propagates per-destination encrypted tokens in `ABC-Auth-Token` headers, and Part B §3.5 names a `URL4-Auth-Token` with a target type. The SDK defers all of it: `url4 serve` ships no authn or authz and asks for a reverse proxy in front. The Engine already has the shape we want: the App mints a per-run capability token and the Runner only carries it.

The idea to test: **the node, through its evaluator, is the only party that holds, validates and issues credentials.**

Endpoints never see a raw credential. An endpoint receives a node-issued session, a capability scoped to one run (`rid`, request tree, purpose, expiry), and uses it for whatever it needs: reading `@` holdings, calling a sibling endpoint, asking the node to fetch an `s3://` source. Outbound, the node forwards the requestor's per-destination tokens and holds the ones addressed to it (Part H §31.5); inbound, the node validates before any endpoint runs. A standalone serverless function is its own node and validates for itself, so the rule holds at every size. The draft already fixes the token's bindings: `rid`, a timestamp with a window of at most 300 seconds, and the destination identity (Part H §31.2).

Questions to answer before this becomes a section:

1. **Session shape.** Is the endpoint-side session the Engine's JWT topic capability generalised (`sub` = run, `iat` window), or the spec's `URL4-Auth-Token`? One token type, or a node-internal one plus the spec's wire one?
2. **Where the identity lives.** `@alice` access control and consent (Part B §5.6.4) need the requestor's identity at the endpoint. Does the session carry it, or does the endpoint ask the node? Part H §33.1 asks the mirror question, whether identity is per server or per endpoint; "per node" is our answer.
3. **Proxy mounts.** For `/claude → url4://beta.example/claude`, which side validates the requestor, and does beta see our identity or a delegated one (Part H §31.2 encrypts to the destination)?
4. **Token destination.** Part H §31 addresses tokens to a node and matches on authority (§31.5). Is the destination always the node, never an endpoint path?
5. **Agent sessions.** The coordinating node issues the `Session-Id` (Part G §28.5). Is that session bound to the run capability, and does the transcript URL it exposes fall under the same access control?
6. **What an endpoint may do with a session.** Only call back into its own node, or reach other nodes directly with a node-minted, destination-bound token?

Recommendation to explore first: node-issued run capability for endpoints (the Engine's pattern), spec §22 tokens between nodes, identity carried in the capability claims. Attribution, consent and audit (Parts E and H) then attach to the same run identity.

A plan endpoint on the node (question, reopened). Deferred in the design session; reopened because three later additions give it inputs it did not have: per-endpoint `accepts` and `emits` (§8), per-endpoint `delivery` and `schemes` (§5, §2), and a node egress that already resolves every target's policy before spending (§11). Today the SDK's `Graph.validate()` checks syntax only and the Engine's preflight checks routes on one node.

The idea: a **dry run** on the node. The draft already obliges an endpoint to do most of it: compute the consumer set by static analysis before resolving anything (Part H §29.2.1), be ready to show a source the full call tree (Part H §30.2), and estimate before executing under a hard budget (Part E §24). Steps 3 and 4 of §11 without step 5: parse, resolve mounts, type-check every edge against `accepts / emits`, check schemes and delivery modes, consult policy and budgets, and return the request tree with per-source verdicts and an estimated cost. Spend nothing. The answer is the envelope (Part D §17) with `sources[]` and `meta` filled in and no `result`; a refused source carries its error code, so "can this node run my expression?" becomes one call with a structured no.

Questions:

1. **Shape.** `Prefer: dry-run` on the ordinary GET (the RFC 7240 pattern we already use for `respond-async`), a `plan` protocol parameter beside `q=`, or a separate path? The header keeps the address identical for plan and run, which suits caching and audit.
2. **Does a plan spend?** Policy-registry consults and budget reservations are real calls. Is a plan free by definition, with `meta.total_cost` as an estimate from `pricing_version`, or may it reserve?
3. **Is the plan an artifact?** A plan id returned as a handle, so a later run can say "execute this plan" and skip re-planning. That ties to idempotency (§7) and to the token binding of §11.
4. **Federation.** For a remote subtree, does the node forward `Prefer: dry-run` to the other node and merge its plan, or plan only from that node's capabilities document? The first is exact and costs a round trip; the second is instant and approximate.
5. **Ownership.** The Engine's preflight admission (OME-880) becomes this endpoint's single-node case. Does the SDK gain `url4 plan`, calling the same thing?

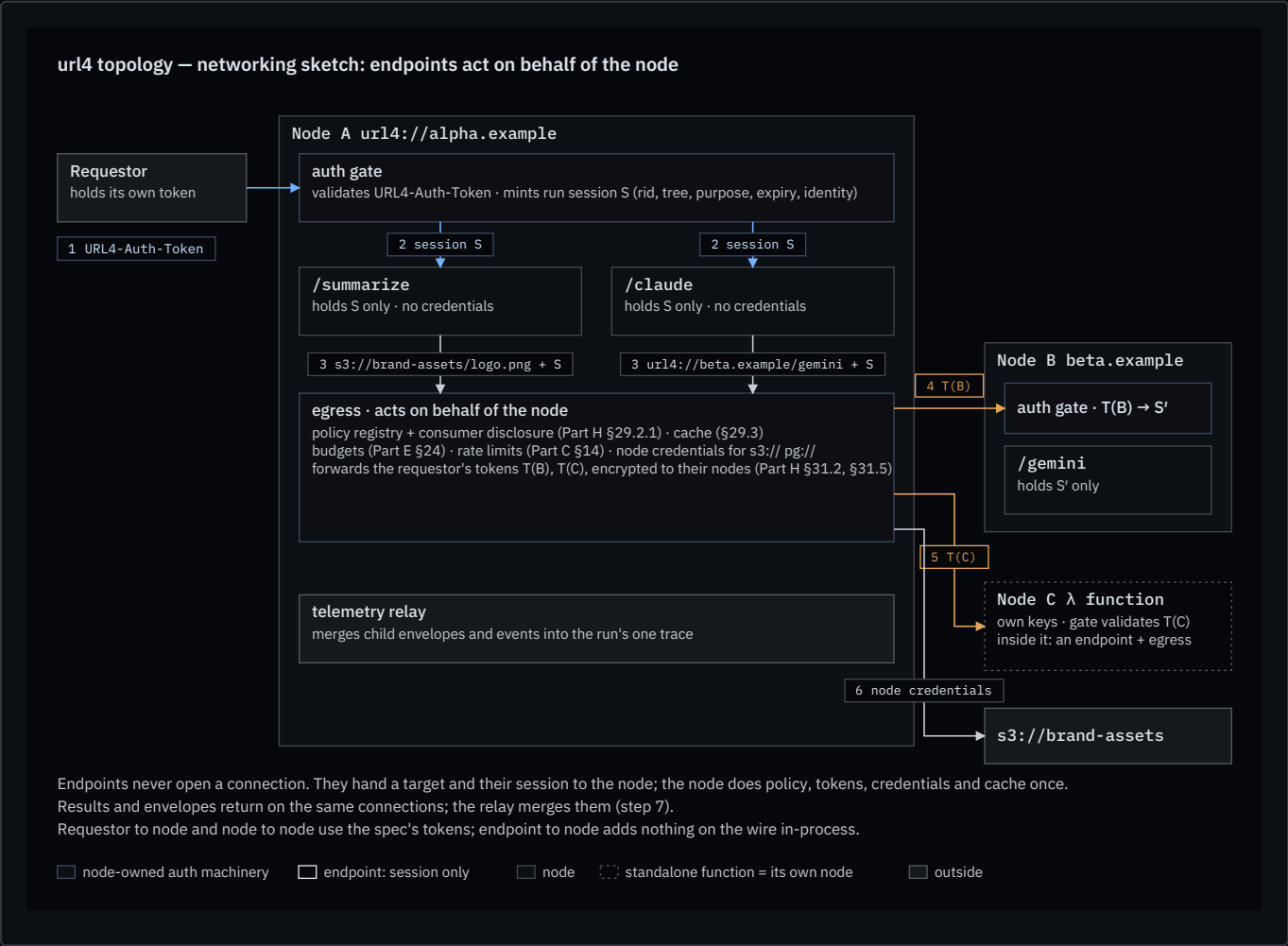
Recommendation to explore first: `Prefer: dry-run` on the same GET, envelope-only answer, cacheable like any GET, forwarded to remote nodes that advertise it and approximated from capabilities where they do not.

Deferred

- **Swarm** as a protocol noun. Would need membership and trust rules that belong to the unwritten governance parts.
- **Endpoint grades** (processor-only vs evaluator). Rejected: every endpoint evaluates.

11. Networking sketch: endpoints act on behalf of the node

A brainstorm, not a decision. It answers one question from §10 concretely: **how does an endpoint make an outbound request without holding a credential?**



Three candidate mechanisms

	A · NODE EGRESS	B · DELEGATED TOKEN	C · SIDECAR
Who opens the outbound connection	the node	the endpoint, with a node-minted token	a per-endpoint proxy process
Where credentials live	node only; it forwards the requestor's destination-encrypted tokens and holds those addressed to it (Part H §31.5)	the requestor's destination-bound token (Part H §31.2) carried by the endpoint	sidecar only
Policy, disclosure, cache, budgets, rate limits	one place: the node's egress (Part H §29.2.1, §16.3, §20, §31)	at mint time on the node; enforcement split	in the sidecar
Serverless endpoint	nothing to configure	needs network egress and key handling per function	platform-dependent
Cost	an extra hop; large payloads through the node, or by reference (§8)	none	one process per endpoint
What it is today	how the SDK already works: an in-process endpoint fetches through the node's <code>IOLayer</code>	the Engine's per-run JWT, generalised	not built

Recommendation to explore first: A for endpoints, B between nodes. An endpoint never talks to the network. It talks to its node. The node talks to other nodes with the spec's tokens. A standalone serverless function is its own node, so the same two rules cover it: the parent node reaches it node-to-node (B), and inside it the function is an endpoint using its own node's egress (A). C is a deployment shape of A, not a third protocol.

The flow the diagram draws

1. The requestor calls Node A with its `URL4-Auth-Token` (Part B §3.5). Node A's auth gate validates it.
2. The gate mints a **run session S**: `rid`, the request tree, purpose, expiry, the requestor's identity. Every endpoint in the run receives S and nothing else. In-process endpoints get it as an object; command mounts get it in the environment.
3. An endpoint that needs `url4://beta.example/gemini` or `s3://brand-assets/logo.png` hands the target and S to the node's **egress**. It does not open a connection.
4. Egress does the node's work once, in one place: consults the policy registry and discloses consumers (Part H §29.2.1), checks budgets and rate limits (Part E §24, Part C §14), serves from cache when allowed, keyed by requestor identity and consumer set so a shared cache is not a policy bypass (Part H §29.3), and either fetches with the node's own credentials (`s3://`, `pg://`) or forwards the requestor's token **T(B)**, encrypted to Node B, on the sub-request (Part H §31.2, §31.3). The node mints nothing for other nodes: the spec has the originator encrypt every token so that intermediaries cannot read or replace them. What the node mints is internal: the run session S.
5. Node B's gate validates T(B), mints its own session S', and runs `/gemini`. Its result and envelope come back on the same connection.
6. A proxy-mounted standalone function is Node C: same as step 5 with T(C).
7. Node A's telemetry relay merges what came back into the run's one trace (doctrine F1).

What is new on the wire, and what is not

- Requestor to node, and node to node: nothing new. The spec's tokens and `traceparent`.
- Endpoint to node: **nothing on the wire for in-process endpoints**; it is a function call through the node's `IOLayer`, which is what `Url4Node` does today. For out-of-process endpoints (command mounts, containers) an explicit node endpoint is needed, something like `GET /.node/egress?u=<absolute URI>` with `URL4-Session: S`. Its path and shape are open.

Open questions this sketch adds

- Does egress return bytes to the endpoint, or a reference (§8)? For large sources, by reference keeps the node out of the data path.
- The spec binds T(B) to `rid`, a timestamp (≤ 300 s) and the destination (Part H §31.2). Does the node-internal session S need the same three bindings?
- Delegated credentials only target `abc_node` or `http_source` (Part H §31.4). For `s3://` and `pg://` the node's own credentials are the only path; should `target_type` widen, or is node-credential-only the rule?
- Can an endpoint ever be granted direct egress (mechanism B at endpoint level)? Probably only for trusted local mounts, and only by node policy.
- Where does the egress endpoint live for command mounts: a Unix socket, a loopback port, or the node's public origin with S as the credential?

12. Alignment with the url4-refactor drafts (2026-09-08)

The `url4-refactor` branch of `OpenMined/screamingface-design` carries draft Parts C–I (cut 2026-04-28 from the v0.4 text, not yet reviewed) and a v0.4 monolith that rennumbers everything after §20. This document was first written from the v0.2 monolith. Every citation has been re-anchored to the Part numbering. The table records what the drafts changed, and how this document resolved it.

TOPIC	WE WROTE	THE DRAFT SAYS	RESOLUTION
Capabilities document	schema unwritten	Part G §27.2 specifies it: one <code>node</code> , <code>collections</code> , <code>intent_processors</code> , <code>processor_policy</code> , <code>mode_support</code>	§5 rewritten as spec shape + marked additions (<code>path</code> , <code>delivery</code> , <code>schemes</code> , <code>default_processor</code>)
Discovery mechanisms	<code>.well-known</code> vs OPTIONS	<code>.well-known/abc-capabilities</code> SHOULD, <code>Capabilities</code> header MAY (Part G §27.1); no OPTIONS	header added as mechanism C; recommendation moved to A + C
Sub-paths	<code>/name</code> is an endpoint	sub-paths are collections (Part G §27.4); processors are addressed by <code>id</code>	proposed <code>intent_processors[].path</code> ; vocabulary realigned to the spec's Node/Endpoint (2026-09-08)
Mounts vs delegation	<code>proxy</code> forwards a sub-request	<code>processor=</code> delegation, five forms, <code>abc_delegate</code> receives materialised sources (Part G §27.3)	mounts mapped onto processor types (§4); no new mechanism
Async	<code>Prefer: respond-async</code> , <code>Location</code> handle	<code>delivery=async</code> ; 202 body with <code>poll_url</code> (Part C §9.1, §12.5)	adopted; <code>Prefer</code> and <code>Location</code> become aliases
Wrapper switch	<code>Accept: application/json</code> = envelope	<code>Accept</code> negotiates the result's type; envelope is always JSON (Part C §9.1, Part F §25.2)	dedicated <code>application/url4-envelope+json</code> (§6)
Ladder	WS → SSE → sync, ours	two edges only (Part C §11.4); degrade-not-fail is a rule (Part C §10.2)	kept as delta, anchored in §10.2
<code>;accept</code> values	MIME types	short aliases only; registry has no image/audio/video rows (Part F §25.5)	examples fixed; alias rows proposed
Binary results	<code>result.media_type</code>	<code>result.content_type</code> ; data URI in LLM mode, raw for RDS (Part G §26.2.3)	adopted; <code>result.artifact</code> stays a delta
<code>ct_mismatch</code>	fail / convert / pass	four values, default <code>ignore</code> , only with <code>;accept</code> (Part G §26.3)	adopted
Telemetry events, cost event	ours	21 SSE types (Part C §12.5); <code>total_cost</code> scalar, <code>budgets_spent</code> (Part D §17, Part E §24)	kept as additions, roll-up mapped to <code>total_cost</code>
Statelessness	every endpoint keeps nothing	the coordinating node owns an agent session and transcript (Part G §28)	§3: state on the node, endpoints stay stateless
Node-minted tokens	egress mints T(B)	originator encrypts every token; intermediaries forward or hold (Part H §31.2, §31.5)	§11 corrected: node forwards, mints only its internal session
Non-HTTP credentials	node's own	<code>target_type</code> is <code>abc_node</code> <code>http_source</code> only (Part H §31.4)	stated; question for Kevin
Artifacts and proxies	free to forward	flow constraints and redistribution depth (Part H §29.2.2)	cautions added to §4 and §8

TOPIC	WE WROTE	THE DRAFT SAYS	RESOLUTION
Root /	ours	unaddressed; examples use bare <code>abc://node?q=</code> (Part I §41.27); <code>v</code> param outranks the path (Part D §19.4)	kept; tensions listed in §2
Cancel	<code>DELETE</code> on <code>Location</code>	<code>DELETE <poll_url></code> or <code>cancel=<rid></code> (Part C §16.2); <code>cancelled</code> not in the status enum (Part D §17.4)	<code>DELETE <poll_url></code> ; enum gap flagged
Naming	<code>url4://</code> , <code>URL4-*</code>	C-I still <code>abc://</code> , <code>ABC-*</code> ; consolidation is open question 19 (Part I §43)	kept <code>url4</code> ; assumption stated in §1

Confirmed by the drafts without change: the three trust relationships and token bindings (Part H §31), the strict request tree (Part H §29.1), `/name` ensembles on one node (Part I §41.9), a weighted image source (Part I §41.20), consumer disclosure before resolution (Part H §29.2.1), the idempotency wording we dispute (Part C §15.1), and every element of our cancel shape (Part C §16.2).

13. url4 as a network protocol: submit, not call (question)

Raised 2026-09-08. Today a url4 expression is called: one requester sends a GET to one node and waits. The question is whether the same expression can be submitted to a network that resolves it over time, the way a torrent is fetched or a transaction is mined: every requester is a node, capable nodes claim the endpoints an expression names, and the result finds its way back.

What the spec already gives such a network

- **The expression is the address, and it can be content-addressed.** `urn:sha256:` is the accepted form for content addressing (Part I §43, question 1). The canonical rendering of an expression hashes to a stable work id.
- **Submit and collect are already decoupled.** `delivery=async` returns a handle and the result arrives later (Part C §11.3); `resume=<rid>` continues from the last known state (Part C §9.1); results carry `result_version` and `is_final`, so an answer may improve over time (Part C §12.5).
- **Redundant evaluation has a vocabulary.** Quorum and triggers already say how many of N must succeed before an answer counts (Part C §12). Run the same endpoint on several nodes and quorum becomes agreement.
- **Discovery can be a record.** A node's capabilities document (Part G §27.2) is a self-description that can live in a distributed table as well as at `/.well-known`.
- **Every requester is a node.** A local SDK process that evaluates is a node in this document already (§4).
- **Attribution is an incentive model.** Attribution scores and the still-undefined settlement (Part E, Part I §37) are where a Bitcoin-like reward would attach.

What is genuinely new

- **Claiming work.** Which node evaluates a submitted expression, and how two nodes avoid doing the same work, or are made to on purpose.
- **Verifying results without determinism.** A torrent piece is verified by hash; an LLM result cannot be. Candidates: agreement across redundant evaluations (quorum), attested execution (Part I §38), reputation, or accepting unverifiable results for some intents.
- **Consent in an open swarm.** A source may name its permitted consumers and cap redistribution depth (Part H §29.2.2); an anonymous claimer is neither known nor permitted. Either sources opt into open resolution, or the swarm is a closed membership.

- **The context travels.** Submitting an expression means its resolved sources travel to whichever node claims it, the opposite of the "code to the data" posture of RDS mode (Part A §1.1).
- **Liveness and economics.** Who guarantees an expression is ever resolved, and who pays. Part I §37 is TBD.

A sketch to argue with. The requester canonicalises the expression and publishes `urn:sha256:<hash>` with the expression to the network. Nodes whose capabilities cover an endpoint the expression names claim that endpoint and evaluate it against their own holdings and credentials, as §11 describes for one node. Each returns a signed envelope keyed by the hash. The requester's `poll_url` becomes a lookup of that key; quorum decides when the answer is good enough; `is_final` closes it. Nothing in the grammar changes; the transport changes from point-to-point to publish-and-claim.

Questions

1. Is this a fourth delivery mode (`delivery=network`) or a different transport beneath the same three?
2. Which verification candidate is acceptable for which kind of intent?
3. Can consent and flow constraints be expressed for an open set of claimers, or is membership required?
4. Does the requester's evaluator keep fan-out and reduce, with the network resolving only leaf endpoints? That keeps §11's egress intact and makes the swarm a resolver, not an orchestrator.

Appendix A — proposed spec deltas for Kevin

Each item names the anchor and the change. All are proposals.

1. **Part A §1.4 — endpoint kinds.** Keep Node and Endpoint as defined. Add that an endpoint is bound to an evaluator and an intent processor, and name the binding kinds `local` | `command` | `proxy` (the processor types `internal` / `function`, `code`, `abc_delegate` of Part G §27.3); a proxy declares its target. Confirm that Part H §31 tokens address nodes, never endpoint paths (§10).
2. **Part B §3.1.1, Part D §19 — root.** `url4://node` $\equiv$ `url4://node/`; `/` is the node's default endpoint at the current version; `/v1` is a version alias. Your own examples already do this (Part I §41.27). Also resolve §19.1 (path primary) against §19.4 (`v` outranks path).
3. **Part C §11.2 — bindings.** `delivery=stream` is SSE on the same GET, requested with `Accept: text/event-stream` (§11.2 names SSE but no media type). An endpoint MAY also offer WebSocket by honouring `Upgrade: websocket` on that same GET (`101`); frames carry the same event names. Orthogonal to Part I question 26, which is about `ws://` sources.
4. **Part C §10.2, §11.4 — ladder and floor.** Extend graceful degradation to `delivery`: the endpoint answers with the richest mode it supports, WebSocket $\rightarrow$ SSE $\rightarrow$ sync, in one round trip; sync is the only MUST and `any` $\rightarrow$ `sync` is always legal. An endpoint answering below the ask is a reported degradation (§10.2.2), not an error. Add a `delivery` list per processor to the capabilities document.
5. **Part C §15.1 — idempotency.** Replace "the protocol does not guarantee idempotency" with: GET is idempotent per RFC 9110 §9.2.2; results are not guaranteed deterministic. §15.2's `budget_exceeded` row stays as the stated exception.
6. **Part C §16 — cancellation.** Take your first option: `DELETE <poll_url>; drop cancel=<rid>`. Terminal state `cancelled`; SSE event `request.cancelled`; propagates to in-flight children; partial result when quorum was met. Add `cancelled` to the status enum in Part D §17.4 and to the state machine in Part C §13.4.
7. **Part D §17 — envelope for sync callers.** A dedicated media type, `application/url4-envelope+json`, selects the envelope; any other `Accept` returns the bare result in its negotiated type. Envelope default `meta=summary` (override of §9.1's `none`). At `meta=full` the envelope carries a `telemetry` block (`logs[]`, `spans[]`, `cost`), present and `null` at `summary` per the Hybrid Rule (§17.2.2), redactable per §18.4. `total_cost` is the roll-up of a separate `cost.usage` stream event.

8. **Part F §25, Part G §26 — typed payloads.** Add image, audio and video rows with aliases (`png` , `jpeg` , `wav` , `mp4`) to the §25.5 registry; extend §26.2.3's data-URI rule to results (`result.content` + `result.content_type`); add `result.artifact` for by-reference results above an endpoint-declared `inline_max_bytes` , subject to §29.2.2 flow constraints; WebSocket binary frames MAY carry bytes, SSE goes by reference. Part E: say how weights and budgets apply to non-text sources. Slot §41.24 is free for a multimodal example.
9. **Part B §3.5 — other schemes.** Generalise the `s3://` row: any scheme other than `url4://` , `https://` , `http://` is a read through a scheme adapter on the evaluating node, resolved with the node's own credentials, typed by the adapter, advertised in the capabilities document (`schemes`), and otherwise `unsupported_mode` (permanent). Path semantics per scheme are the adapter's contract.
10. **Part G §27.2 — capabilities document.** Keep the schema; add `intent_processors[].path` so a processor is addressable as an endpoint, `delivery` per processor, `schemes` the node reads (§2), `inline_max_bytes` , and a `default_processor` . Rename `abc_*` to `url4_*` with the header consolidation (your question 19).
11. **Part H §31.4 — credential targets.** Either widen `target_type` beyond `abc_node` | `http_source` , or state that non-HTTP schemes are resolved with the node's own credentials only (§2).
12. **Part D §17.2.2, §20 — additions must respect the Hybrid Rule and propagated.** Our `telemetry` and `session` blocks are present-and-null at `summary` ; unknown fields pass through unaggregated.

Appendix B — follow-up work items

To file after owner review, one per landing.

LANDING	ITEM
screamingface-engine	Node surface phase 1: discovery per §5, <code>GET /?q=</code> with SSE, <code>DELETE</code> on the run handle
screamingface-engine	Model endpoints reachable over HTTP (phase 2); standalone model functions, proxy-mounted (phase 3)
screamingface-engine	Rename the <code>URL4-Capability</code> JWT header to avoid the "capabilities" collision
url4-sdk	Delivery negotiation in <code>Url4Node.asgi()</code> : <code>Upgrade / Accept</code> handling, <code>application/url4-envelope+json</code> , SSE body, <code>delivery=async</code> with <code>poll_url</code> , optional WebSocket answer; evaluator-side single-request ladder in <code>HttpIOLayer</code>
url4-sdk	Capabilities document and/or OPTIONS responder, after the §5 decision; advertise <code>delivery</code> per endpoint
url4-sdk	<code>url4 serve</code> default path <code>/</code> ; <code>Client</code> default path <code>/</code> ; <code>/v1</code> alias
url4-sdk	<code>proxy mount</code> kind in <code>url4.toml</code>
url4-sdk	Scheme adapters (<code>s3://</code> , <code>pg://</code> , <code>sqlite://</code> , ...) as <code>IOLayer</code> adapters registered by scheme; <code>[schemes]</code> in <code>url4.toml</code> ; <code>schemes</code> in capabilities
url4-sdk	Typed payloads: bytes + media type through <code>IOLayer / FetchRequest</code> , <code>;accept / ct_mismatch</code> enforcement, <code>result.artifact</code> by-reference fetch
screamingface-engine	Non-text processors over aigateway (image, speech); artifact store as the by-reference path; <code>accepts / emits</code> in capabilities
url4-sdk	<code>Url4Node</code> → node naming retrofit with a deprecation alias
repo	Doctrine skill synced in this unit (T1, N1, F2, F4, term table)

LANDING	ITEM
Kevin	Review Appendix A

Appendix D — Vocabulary

One line each: what it is, why it matters, where the spec grounds it. Terms marked ours are proposals in this document.

Language

TERM	DEFINITION	ANCHOR
Expression	<code>(sources)!</code> <code>intent</code> : given this data, do this. The atomic unit of work and the whole composition; sources may be expressions, so expressions nest into a tree.	Part A §1.4, Part B §2
Source	One input: inline text, a URI, or a nested expression. Carries attribution annotations (<code>name:weight:budget</code>) and execution annotations (<code>!t</code> , <code>!retry</code> , <code>!accept</code>).	Part B §4
Intent	The right side of <code>!</code> : a prompt, a code pointer, a relative or remote URI, or a computed expression.	Part B §6
Intent processor	What turns resolved context plus intent into a result: a model call, a script, a command, or a delegate on another node.	Part A §1.4, Part G §27.3
Holdings, <code>@</code>	The endpoint's own data via the self-reference token; <code>@alice</code> names a principal's data under access control and consent.	Part B §5.6
Collection	A source that parses into rows; <code>*</code> iterates it. A sub-path after an endpoint also selects a collection.	Part B §5.3, Part G §27.4

Topology

TERM	DEFINITION	ANCHOR
Endpoint	A stateless function at a path (<code>/claude</code>). Answers <code>GET <path>?q=<expr></code> , evaluates it, returns a result. One grade: every endpoint evaluates url4. The spec's word.	§3, Part A §1.4
Node	The origin that serves a set of endpoints, <code>/</code> being the default, publishes discovery, owns credentials and outbound traffic. Every requester running an evaluator is one. The spec's word.	§4, Part A §1.4
Mount	The binding of a path to an endpoint implementation: <code>local</code> , <code>command</code> , <code>proxy</code> . The spec's processor types <code>internal / function</code> , <code>code</code> , <code>abc_delegate</code> .	§4, Part G §27.3
Evaluator	The code that runs an expression: resolves sources, fans out, reduces, runs the intent. Every endpoint contains one.	§1
Requestor	Whoever submits an expression.	Part A §1.4
Request tree	The nodes and endpoints one expression touches; strictly a tree. No plan object, no swarm.	Part H §29.1
Capabilities document	JSON a node publishes at <code>/.well-known/url4-capabilities</code> : processors, collections, delivery modes, schemes. A <code>Capabilities</code> header may point to it.	Part G §27.1, §27.2
Scheme adapter (ours)	A node-mounted reader for a non-url4 scheme (<code>s3://</code> , <code>pg://</code> , <code>sqlite://</code>) with the node's own credentials, typing its result, advertised in capabilities.	§2, Part B §3.5

Transport

TERM	DEFINITION	ANCHOR
Delivery mode	How the answer returns: <code>sync</code> , <code>stream</code> (SSE on the same GET), <code>async</code> (202 + <code>poll_url</code>), and our WebSocket rung (<code>101</code> on the same GET). Sync is the only MUST.	§6, Part C §11
Ladder (ours)	The endpoint answers with the richest mode it supports, WebSocket → SSE → sync, in one round trip; answering below the ask is a reported degradation.	§6, Part C §10.2
Envelope	The JSON wrapper: <code>result</code> , <code>status</code> , <code>delivery</code> , <code>sources[]</code> , <code>meta</code> . Requested with <code>Accept: application/url4-envelope+json</code> ; otherwise the bare result.	Part D §17
<code>meta</code>	Envelope depth: <code>none</code> , <code>summary</code> (counts, latency, cost), <code>full</code> (per-source detail, nested envelopes, telemetry).	Part D §17.2
Run handle , <code>poll_url</code>	The address of an async run: <code>GET</code> for status, <code>DELETE</code> to cancel.	Part C §12.5, §16
<code>rid</code>	The request id; fresh per child request, recorded by the parent, bound into tokens.	Part C §9.1
Telemetry signals	Logs, spans (tokens live here), and <code>cost.usage</code> events (money lives here). In-band in every mode; OTLP is the durable copy.	§7

Data

TERM	DEFINITION	ANCHOR
Typed payload (ours)	Every edge carries a value named by its media type; text is the default. Sources declare by <code>Content-Type</code> , results by <code>result.content_type</code> .	§8
Artifact (ours)	A large result returned as an <code>https://</code> URL the next endpoint fetches as data, above an endpoint-declared <code>inline_max_bytes</code> ; inherits flow constraints.	§8
<code>;accept</code> , <code>ct_mismatch</code>	A source's wanted format, as a short alias, and what to do when the fetched type differs: <code>fail</code> , <code>ignore</code> , <code>transform_ignore</code> , <code>transform_fail</code> .	Part F §25.6, Part G §26.3

Trust

TERM	DEFINITION	ANCHOR
Run session (ours)	A node-issued capability scoped to one run (<code>rid</code> , tree, purpose, expiry, identity) that endpoints carry instead of credentials.	§10, §11
Inter-node token	The requestor's credential for one node, encrypted to that node's key, bound to <code>rid</code> , a timestamp (≤ 300 s) and the destination; intermediaries forward what they cannot decrypt.	Part H §31
Egress (ours)	The node component that performs every outbound request for its endpoints: policy, disclosure, cache, budgets, credentials, token forwarding.	§11
Policy registry	The out-of-band service a source exposes to state its terms; consulted before resolution.	Part H §30
Flow constraints	A source's limits on where its output may travel: permitted or denied consumers, redistribution depth.	Part H §29.2.2
Attribution	The per-source influence score the envelope reports; weights and budgets shape it.	Part E

Execution

TERM	DEFINITION	ANCHOR
Quorum, trigger	Quorum: how many sources must succeed before the intent may run. Trigger: the terminal-source count at which the endpoint decides whether to produce a result.	Part C §12
Degradation	An endpoint must fall back rather than fail when it cannot honour a request, and must report it.	Part C §10.2
Agent session	A multi-turn interaction among <code>mode=agent</code> sources coordinated by the resolving endpoint, which holds the transcript; <code>coord=</code> selects the mode.	Part G §28
Plan, dry run (ours)	A node endpoint that resolves and type-checks an expression, consults policy and budgets, and returns the envelope with no result. Open question.	§10
Idempotent	A url4 GET has no extra server-side effect on repeat (RFC 9110); results need not be identical. Disputed wording in the spec.	Part C §15